```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.cluster import dbscan
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report,confusion_matrix,accuracy_score
from sklearn.datasets import load_iris
```

```
iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

X
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

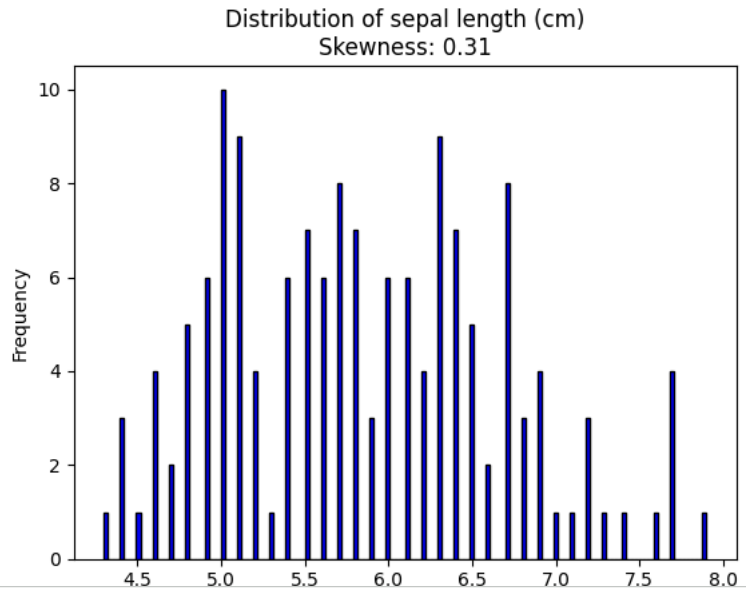
150 rows × 4 columns

Next steps: Generate code with X | New interactive sheet

```
print (iris["target_names"])
```

```
['setosa' 'versicolor' 'virginica']
```

```
for col in X.columns:
    plt.figure()
    plt.hist(X[col], bins=150, color='blue', edgecolor='black')
    plt.title(f'Distribution of {col}\nSkewness: {X[col].skew():.2f}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()
```




```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

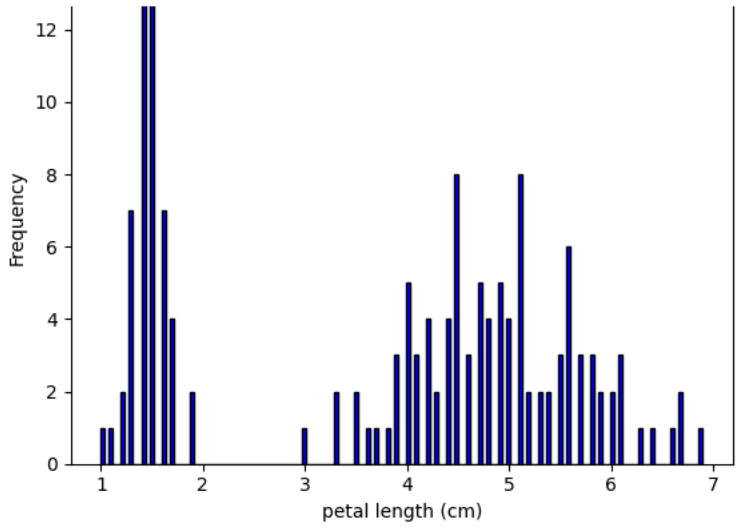
sc_unsupervised = StandardScaler()
X_scaled = sc_unsupervised.fit_transform(X)

sc_supervised = StandardScaler()
X_train_scaled = sc_supervised.fit_transform(X_train)
X_test_scaled = sc_supervised.transform(X_test)
```

Kmeans

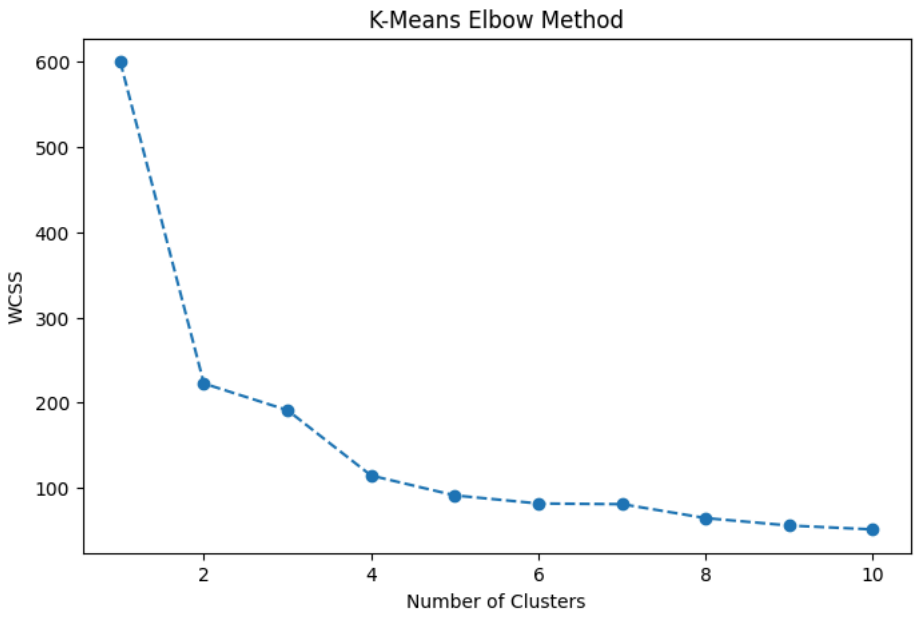
```
wcss = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42,)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(K_range, wcss, marker='o', linestyle='--')
plt.title('K-Means Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
#optimal number = 5
```



Distribution of petal width (cm)

Skewness: -0.10



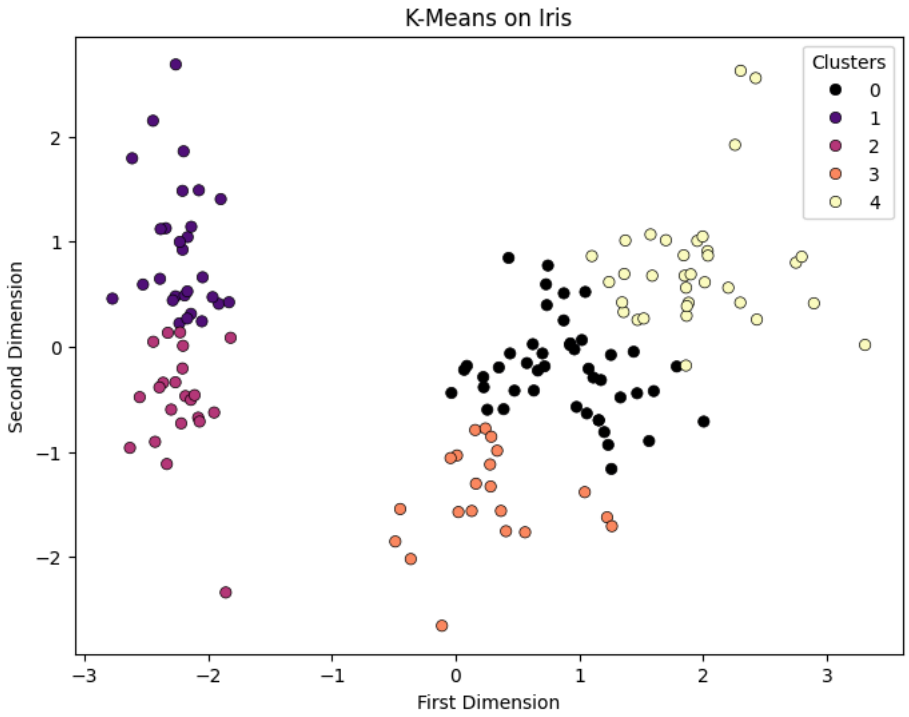

```
kmeans_model = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans_model.fit_predict(X_scaled)
```

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
sns.scatterplot(
    x=X_pca[:, 0],
    y=X_pca[:, 1],
    hue=y_kmeans,
    edgecolor='k',
    palette='magma',
)

plt.title('K-Means on Iris')
plt.xlabel('First Dimension')
plt.ylabel('Second Dimension')
plt.legend(title="Clusters")
plt.show()
```

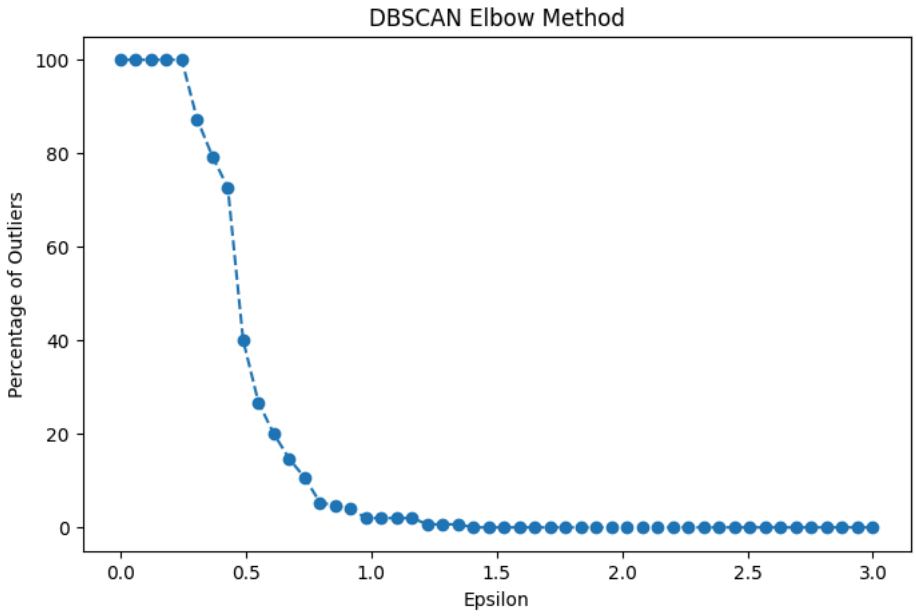


Dbscan

```
from sklearn.cluster import DBSCAN

outlier_percent = []
eps_values = np.linspace(0.001, 3, 50)
for eps in eps_values:
    dbscan_model = DBSCAN(eps=eps, min_samples=2 * X_scaled.shape[1])
    dbscan_model.fit(X_scaled)
    perc_outliers = 100 * np.sum(dbscan_model.labels_ == -1) /len(dbscan_model.labels_)
    outlier_percent.append(perc_outliers)

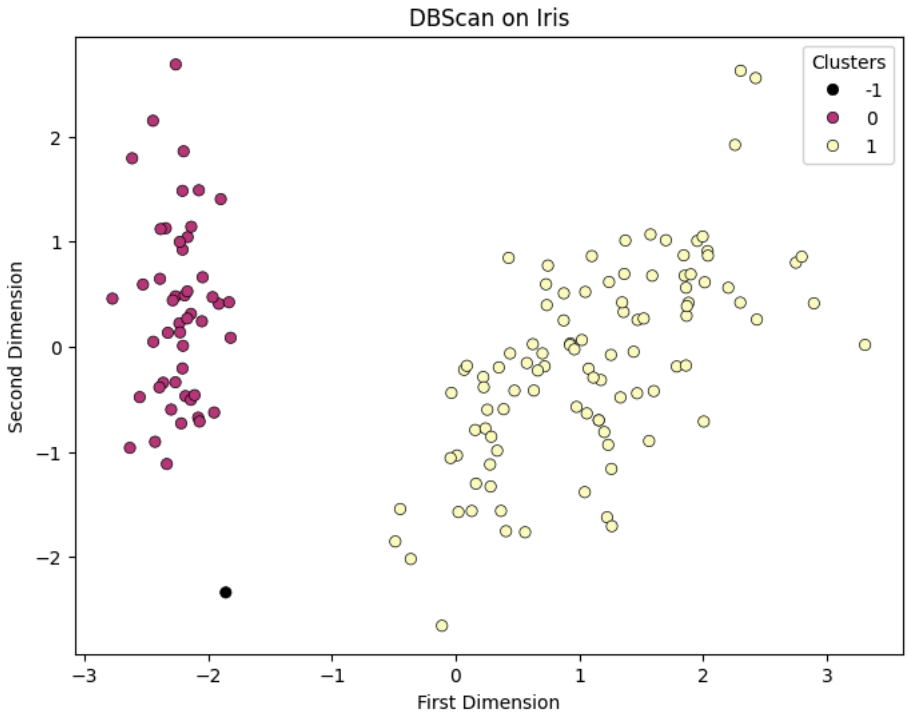
plt.figure(figsize=(8, 5))
plt.plot(eps_values, outlier_percent, marker='o', linestyle='--')
plt.title('DBSCAN Elbow Method')
plt.xlabel('Epsilon')
plt.ylabel('Percentage of Outliers')
plt.show()
#Optimal Epsilon = 1.25
#Optimal Minimum Number Of Points = 2 * Number Of Features
```




```
dbscan_model = DBSCAN(eps=1.25, min_samples= 2 * X_scaled.shape[1])
y_dbscan = dbscan_model.fit_predict(X_scaled)

plt.figure(figsize=(8, 6))
sns.scatterplot(
    x=X_pca[:, 0],
    y=X_pca[:, 1],
    hue=y_dbscan,
    edgecolor='k',
    palette='magma',
)

plt.title('DBScan on Iris')
plt.xlabel('First Dimension')
plt.ylabel('Second Dimension')
plt.legend(title="Clusters")
plt.show()
```

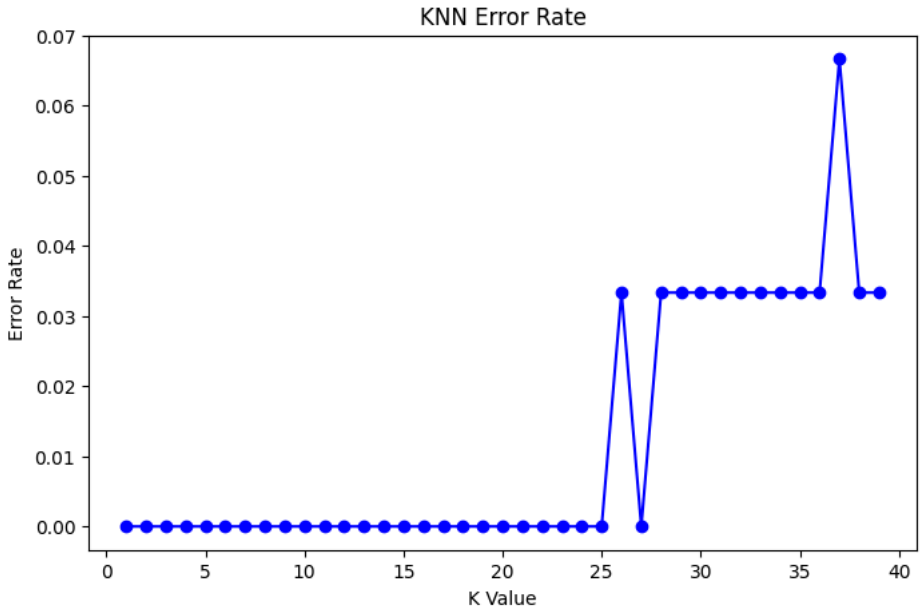


KNN

```
error_rates = []
K_range_knn = range(1, 40)

for i in K_range_knn:
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train_scaled, y_train)
    pred_i = knn.predict(X_test_scaled)
    error_rates.append(np.mean(pred_i != y_test))

plt.figure(figsize=(8, 5))
plt.plot(K_range_knn, error_rates, color='blue', marker='o')
plt.title('KNN Error Rate')
plt.xlabel('K Value')
plt.ylabel('Error Rate')
plt.show()
```




```
#Resizing the test data

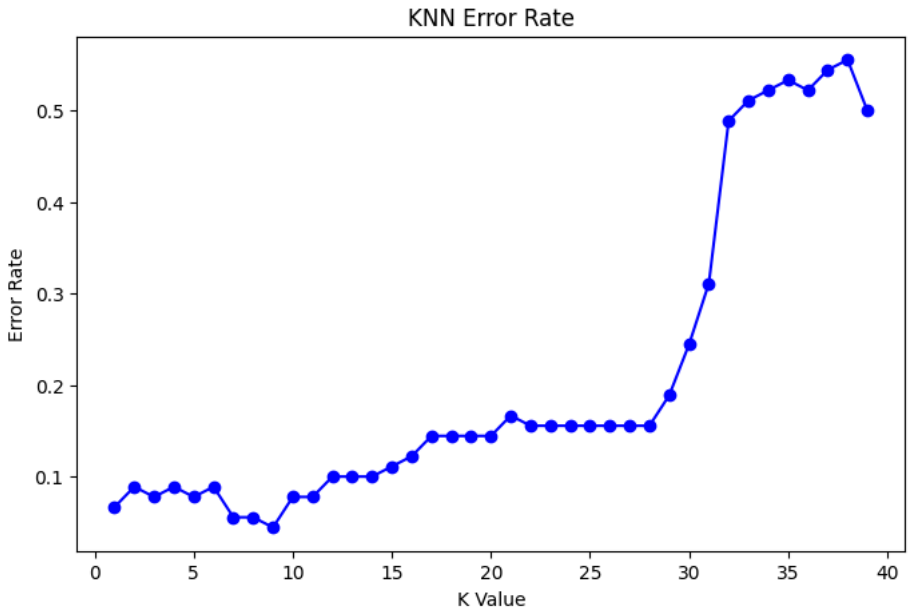
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.6, random_state=42)

X_train_scaled = sc_supervised.fit_transform(X_train)
X_test_scaled = sc_supervised.transform(X_test)
```

```
error_rates = []
K_range_knn = range(1, 40)

for i in K_range_knn:
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train_scaled, y_train)
    pred_i = knn.predict(X_test_scaled)
    error_rates.append(np.mean(pred_i != y_test))

plt.figure(figsize=(8, 5))
plt.plot(K_range_knn, error_rates, color='blue', marker='o')
plt.title('KNN Error Rate')
plt.xlabel('K Value')
plt.ylabel('Error Rate')
plt.show()
```

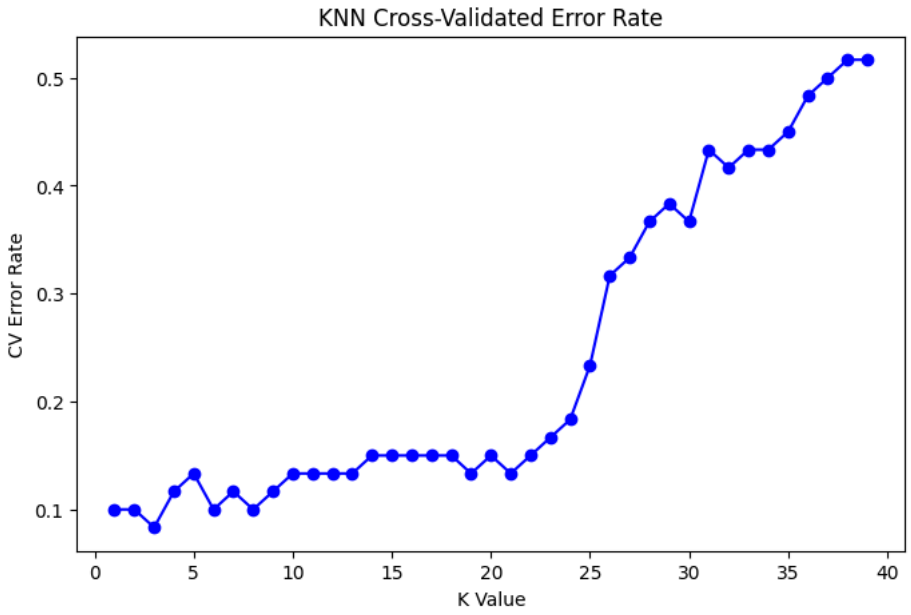



```
from sklearn.model_selection import cross_val_score

cv_error_rates = []
K_range_knn = range(1, 40)

for i in K_range_knn:
    knn = KNeighborsClassifier(n_neighbors=i)
    scores = cross_val_score(knn, X_train_scaled, y_train, cv=5, scoring='accuracy')
    cv_error_rates.append(1 - np.mean(scores))

plt.figure(figsize=(8, 5))
plt.plot(K_range_knn, cv_error_rates, color='blue', marker='o')
plt.title('KNN Cross-Validated Error Rate')
plt.xlabel('K Value')
plt.ylabel('CV Error Rate')
plt.show()
#optimal Solution = 3
```




```
classifier = KNeighborsClassifier(n_neighbors=3)

classifier.fit(X_train_scaled, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

```
y_pred = classifier.predict(X_test_scaled)
print(np.concatenate((y_pred.reshape(len(y_pred), 1), y_test.reshape(len(y_test), 1)), 1))
```

```
[2 2]
[1 2]
[2 2]
[0 0]
[0 0]
[0 0]
[0 0]
[1 1]
[0 0]
[0 0]
[1 2]
[1 1]
[0 0]
[0 0]
[0 0]
[2 2]
[1 1]
[1 1]
[0 0]
[0 0]
[1 1]
[1 2]
[2 2]
[1 1]
[2 2]
[1 1]
[2 2]
[1 1]
[0 0]
[2 2]
[1 1]
[0 0]
[0 0]
[0 0]
[1 1]
[1 2]
[0 0]
[0 0]
[0 0]
[1 1]
[0 0]
[1 1]
[2 2]
[0 0]
[1 1]
[1 2]
[0 0]
[1 2]
[2 2]
[1 1]
[1 1]
[2 2]
[1 1]
[0 0]
[1 1]
[2 2]
[0 0]
[0 0]
[1 1]
```

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
Confusion Matrix:
 [[35  0  0]
 [ 0 28  1]
 [ 0  6 20]]
Test Accuracy: 0.9222222222222223
```

SVM

```
svc_model = SVC() # = svm_model = SVC(kernel='rbf', C=1.0, random_state=42)
```

```
svc_model.fit(X_train_scaled,y_train)
```

```
SVC()
```

```
y_predict = svc_model.predict(X_test_scaled)
```

```
y_pred = classifier.predict(X_test_scaled)
print(np.concatenate((y_pred.reshape(len(y_pred), 1), y_test.reshape(len(y_test), 1)), 1))
```

```
[2 2]
 [1 2]
 [2 2]
 [0 0]
 [0 0]
 [0 0]
 [0 0]
 [1 1]
 [0 0]
 [0 0]
 [1 2]
 [1 1]
 [0 0]
 [0 0]
 [0 0]
 [2 2]
 [1 1]
 [1 1]
 [0 0]
 [0 0]
 [1 1]
 [1 2]
 [2 2]
 [1 1]
 [2 2]
 [1 1]
 [2 2]
 [1 1]
 [0 0]
 [2 2]
 [1 1]
 [0 0]
 [0 0]
 [0 0]
 [1 1]
 [1 2]
 [0 0]
 [0 0]
 [0 0]
 [1 1]
 [0 0]
 [1 1]
 [2 2]
 [0 0]
 [1 1]
 [1 2]
 [0 0]
 [1 2]
 [2 2]
 [1 1]
 [1 1]
 [2 2]
 [1 1]
 [0 0]
 [1 1]
 [2 2]
 [0 0]
 [0 0]
 [1 1]
```

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
Confusion Matrix:
 [[35  0  0]
```